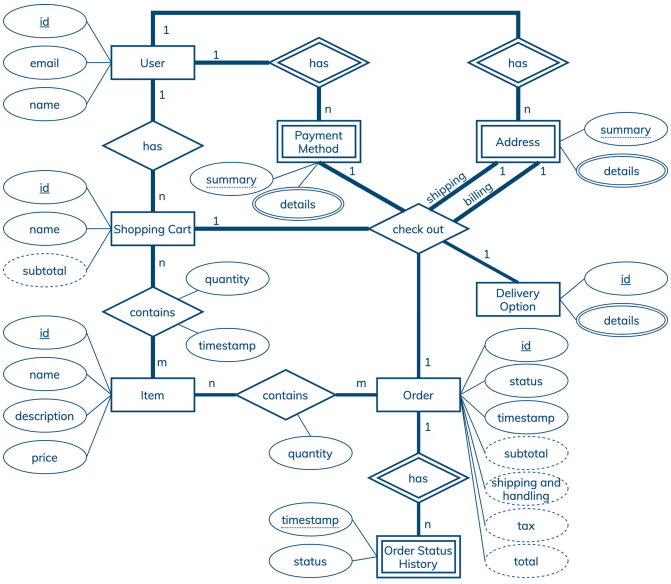
Order Management Data Modeling

Interested in learning more about Cassandra data modeling by example? This example demonstrates how to create a data model for an order management system. It covers a conceptual data model, application workflow, logical data model, physical data model, and final CQL schema and query design.

GET STARTED



Entity-Relationship Diagram

Conceptual Data Model

A conceptual data model is designed with the goal of understanding data in a particular domain. In this example, the model is captured using an Entity-Relationship Diagram (ERD) that documents entity types, relationship types, attribute types, and cardinality and key constraints.

The conceptual data model for order management data features users, payment methods, addresses, items, shopping carts, orders, delivery options, and order statuses. While a user can have many payment methods, addresses and shopping carts, each payment method, address and shopping cart must belong to exactly one user. Each shopping cart can contain many items and each item or, to be more precise, item type can be found in many shopping carts. A checkout process involves one shopping cart, one payment method, one shipping address, one billing address and one delivery option, and results in one order. Each order has a unique id, current status, timestamp, subtotal, shipping and handling fees, tax, and total. The last four attribute types are derived attribute types. For example, an order

subtotal is computed based on prices and quantities of all items in an order. Order items are derived from shopping cart items. Finally, an order status history is maintained to keep track of all order statuses. Since the order status history entity type is a weak entity type, it is uniquely identified by the combination of an order id and order status timestamp. For more information about other attribute types and key constraints, please refer to the diagram.

Next: Application Workflow

Application Workflow

An application workflow is designed with the goal of understanding data access patterns for a data-driven application. Its visual representation consists of application tasks, dependencies among tasks, and data access patterns. Ideally, each data access pattern should specify what attributes to search for, search on, order by, or do aggregation on.

The application workflow has five tasks. The first one is an entry-point task that shows all orders placed by a user. This task is supported by data access pattern Q1. The second task displays information about a selected order, which requires data access pattern Q2. Next, it is possible to go with one of the three remaining tasks. Showing all orders containing a specific item requires data access pattern Q3. Showing an order status history requires data access pattern Q4. Finally, canceling an order requires data access pattern U1.

All in all, there are five data access patterns for a database to support. While Q1, Q2, Q3 and Q4 are used to retrieve data, U1 is intended to update data. In this example, the update pattern is especially interesting because it may require updating rows in multiple tables and may involve race conditions.

Next: Logical Data Model

Logical Data Model

A logical data model results from a conceptual data model by organizing data into Cassandra-specific data structures based on data access patterns identified by an application workflow. Logical data models can be conveniently captured and visualized using Chebotko Diagrams that can feature tables, materialized views, indexes and so forth.

The logical data model for order management data is represented by the shown Chebotko Diagram. Table orders_by_user is designed to support data access pattern Q1. It has multi-row partitions, where each partition corresponds to one user and each row corresponds to one order. Rows within each partition are sorted based on order timestamps. To retrieve all orders placed by a given user, at most one partition needs to be accessed. Next, table orders_by_id covers data access pattern Q2. In this table, each partition corresponds to an order and each row represents an item. Items from the same order are always sorted using their names. The table also has many static columns that describe orders and their associated payment, billing, shipping, and delivery information. Again, this very efficient design requires retrieving only one partition to satisfy Q2. Table orders_by_user_item enables data access pattern Q3. Each partition in this table can store all orders placed by a specific user for a particular item. Finally, table order_status_history_by_id is designed to support data access pattern Q4. Once again, only one partition in this table needs to be accessed to answer Q4.

Note that, in this example, U1 does not directly affect schema design. The diagram simply documents which tables need to be accessed to satisfy this data access pattern. Interestingly, not only U1 accesses three tables that need to be updated simultaneously, order status updates may have additional constraints and race conditions, such as payment processing must happen before an order can be shipped. A sample implementation of this access pattern can be found in the Skill Building section.

Next: Physical Data Model

Physical Data Model

A physical data model is directly derived from a logical data model by analyzing and optimizing for performance. The most common type of analysis is identifying potentially large partitions. Some common optimization techniques include splitting and merging partitions, data indexing, data aggregation and concurrent data access optimizations.

The physical data model for order management data is visualized using the Chebotko Diagram. This time, all table columns have associated data types. It should be evident that none of the tables can have very large partitions. For example, it is unlikely that any single user can place 100000 orders or there could be an order with 100000 different items. This data model is already efficient and scalable, and requires no further optimization. Our final blueprint is ready to be instantiated in Cassandra.

Skill Building

Skill Building

Want to get some hands-on experience? Give our interactive lab a try! You can do it all from your browser, it only takes a few minutes and you don't have to install anything.

Start Hands-on Lab

(A GitHub account may be required to run this lab in Gitpod.
Supported browsers: Firefox, Chrome/Chromium, Edge, Brave.)

More Resources

Review the Cassandra data modeling process with these videos